[40] Correct functioning code to solve the Convex Hull problem using the divide and conquer scheme discussed above. Include your documented source code:

```
def convexHull(self, points):
        #Runs recursive divide and conquer algorithm on points to obtain convex hull points.
        #Uses convex hull points to form a list of lines for said convex hull.
        #Returns convex hull in the form of its outer lines.
        #Time complexity: O(nlog2(n)).
        #Space complexity: O(n).

        points, l, r = self.convexRecurse(points)

        lines = list()
        for i in range(len(points)-1):
                lines.append(QLineF(points[i],points[i+1]))
        lines.append(QLineF(points[len(points)-1],points[0]))

        return lines

def convexRecurse(self, points):
        #Returns if points has size of 1.
        #Splits points into left and right halves and recurses with both of them to get their convex hulls.
        #Calculates common upper and lower tangents with helper methods.
        #Constructs new convex hull using subhulls and tangents with helper method.
        #Returns new convex hull in the form of ordered points.
        #Time complexity: O(nlog2(n)).
        #Space complexity: O(n).

        if len(points)==1:
                return points, 0, 0

        left, lm, rightmost = self.convexRecurse(points[:int(len(points)/2)])
        right, leftmost, rm = self.convexRecurse(points[int(len(points)/2):])

        upperleft, upperright = self.upperConn(left, right, leftmost, rightmost)
        lowerleft, lowerright = self.lowerConn(left, right, leftmost, rightmost)

        newpoints, newleftmost, newrightmost = self.formHull(left, right, upperright, lowerright, lowerleft, upperleft)
        return newpoints, newleftmost, newrightmost

def upperConn(self, left, right, leftmost, rightmost):
        #Starts with left subhull's rightmost point and right subhull's leftmost point as upper tangent.
        #Iterates clockwise on right subhull until highest tangent slope is found.
        #Iterates counterclockwise on left subhull until lowest tangent slope is found.
        #Repeats steps 2 and 3 until no changes are being made.
        #Returns common upper tangent in the form of two points.
        #Time complexity: O(n).
        #Space complexity: O(n).

        upperleft = rightmost
        leftstart = rightmost
        upperright = leftmost
        rightstart = leftmost
        upperslope = (right[upperright].y() - left[upperleft].y()) / (right[upperright].x() - left[upperleft].x())
        anychange = 1
```

```
        while anychange == 1:
                anychange = 0
                if len(right) > 1:
                        i = 0
                        rightindex = upperright
                        rightchange = 1
                        while i < len(right) and rightchange == 1:
                                newindex = (rightstart + i) % len(right)
                                newslope = (right[newindex].y() - left[upperleft].y()) /
                                                        (right[newindex].x() - left[upperleft].x())
                                if newslope >= upperslope:
                                        if rightindex != newindex:
                                                upperslope = newslope
                                                upperright = newindex
                                                anychange = 1
                                        rightindex = newindex
                                        i = i + 1
                                else:
                                        rightstart = newindex
                                        rightchange = 0
                if len(left) > 1:
                        j = 0
                        leftindex = upperleft
                        leftchange = 1
                        while j < len(left) and leftchange == 1:
                                newindex = (leftstart - j) % len(left)
                                newslope = (right[upperright].y() - left[newindex].y()) /
                                                        (right[upperright].x() - left[newindex].x())
                                if newslope <= upperslope:
                                        if leftindex != newindex:
                                                upperslope = newslope
                                                upperleft = newindex
                                                anychange = 1
                                        leftindex = newindex
                                        j = j + 1
                                else:
                                        leftstart = newindex
                                        leftchange = 0
        return upperleft, upperright

def lowerConn(self, left, right, leftmost, rightmost):
        #Starts with left subhull's rightmost point and right subhull's leftmost point as lower tangent.
        #Iterates counterclockwise on right subhull until lowest tangent slope is found.
        #Iterates clockwise on left subhull until highest tangent slope is found.
        #Repeats steps 2 and 3 until no changes are being made.
        #Returns common lower tangent in the form of two points.
        #Time complexity: O(n).
        #Space complexity: O(n).

        lowerleft = rightmost
        leftstart = rightmost
        lowerright = leftmost
        rightstart = leftmost
        lowerslope = (right[lowerright].y() - left[lowerleft].y()) / (right[lowerright].x() - left[lowerleft].x())
        anychange = 1
```

```
        while anychange == 1:
                anychange = 0
                if len(right) > 1:
                        i = 0
                        rightindex = lowerright
                        rightchange = 1
                        while i < len(right) and rightchange == 1:
                                newindex = (rightstart - i) % len(right)
                                newslope = (right[newindex].y() - left[lowerleft].y()) /
                                                        (right[newindex].x() - left[lowerleft].x())
                                if newslope <= lowerslope:
                                        if rightindex != newindex:
                                                lowerslope = newslope
                                                lowerright = newindex
                                                anychange = 1
                                        rightindex = newindex
                                        i = i + 1
                                else:
                                        rightstart = newindex
                                        rightchange = 0
                if len(left) > 1:
                        j = 0
                        leftindex = lowerleft
                        leftchange = 1
                        while j < len(left) and leftchange == 1:
                                newindex = (leftstart + j) % len(left)
                                newslope = (right[lowerright].y() - left[newindex].y()) /
                                                        (right[lowerright].x() - left[newindex].x())
                                if newslope >= lowerslope:
                                        if leftindex != newindex:
                                                lowerslope = newslope
                                                lowerleft = newindex
                                                anychange = 1
                                        leftindex = newindex
                                        j = j + 1
                                else:
                                        leftstart = newindex
                                        leftchange = 0
        return lowerleft, lowerright

def formHull(self, left, right, upperright, lowerright, lowerleft, upperleft):
        #Creates new convex hull in the form of an empty list of points.
        #Adds all of right subhull's points (from upper tangent point to lower tangent point) to hull list.
        #Adds all of left subhull's points (from lower tangent point to upper tangent point) to hull list.
        #Calculates new leftmost and rightmost points by iterating through the new convex hull.
        #Returns new convex hull as a list of points and also returns the new leftmost and rightmost points.
        #Time complexity: O(n).
        #Space complexity: O(n).

        newpoints = list()

        endi = 0
        i = 0
        while endi == 0:
                index = (upperright + i) % len(right)
```

```
                newpoints.append(right[index])
                if (index == lowerright):
                        endi = 1
                i = i+1

        endj = 0
        j = 0
        while endj == 0:
                index = (lowerleft + j) % len(left)
                newpoints.append(left[index])
                if (index == upperleft):
                        endj = 1
                j = j+1

        newleftmost = 0
        newrightmost = 0
        for k in range(len(newpoints)):
                if newpoints[k].x() > newpoints[newrightmost].x():
                        newrightmost = k
                if newpoints[k].x() < newpoints[newleftmost].x():
                        newleftmost = k

        return newpoints, newleftmost, newrightmost

# This is the method that gets called by the GUI and actually executes
# the finding of the hull
        def compute_hull( self, points, pause, view):
                self.pause = pause
                self.view = view
                assert( type(points) == list and type(points[0]) == QPointF )

                t1 = time.time()
                points.sort(key=QPointF.x)
                t2 = time.time()

                t3 = time.time()
                polygon = self.convexHull(points)
                t4 = time.time()

                # when passing lines to the display, pass a list of QLineF objects.  Each QLineF
                # object can be created with two QPointF objects corresponding to the endpoints
                self.showHull(polygon,RED)
                self.showText('Time Elapsed (Convex Hull): {:3.3f} sec'.format(t4-t3))
```

[15] Explain the time and space complexity of your algorithm by showing and summing up the complexity of each subsection of your code:
The common upper tangent function and common lower tangent function both have a time complexity of O(n) because they both go through a maximum of all of the left and right sub-hull points a maximum of one time. The function that builds the new convex hull has a time complexity of O(n) (with k=2) because it goes through all the left and right sub-hull points twice. The recursion function recurses $\log_2 n$ times and houses all three of the aforementioned functions, making its time complexity $O(n\log_2 n)$ (with k=4). The initializer function houses the recursion function and also loops through the returned hull once, making its time complexity $O(n\log_2 n)$ (with k=4) (full equation is $O(4n\log_2 n+n)$, practical full equation is $O(4n\log_2 n)$).
The common upper tangent function and common lower tangent function both have a space complexity of O(n) (with k=0.5) because they both store a maximum of all of the left or right sub-hull points. The function that builds the new convex hull has a space complexity of O(n) because it stores a maximum of all the left and right sub-hull points. The recursion function houses all three of the aforementioned functions, making its space complexity O(n). The initializer function houses the recursion function and also builds the list of lines for the returned convex hull, making its time complexity O(n) (with k=2) (full equation is O(2n)).

Also, include your theoretical analysis for the entire algorithm including discussion of the recurrence relation:
Theoretical analysis yields O(nlogn) (see above).
Recurrence relation: T(n) = 2T(n/2)+O(n). a/(b^d) = 2/(2^1) = 1. T(n) = O(nlogn).
Theoretical analysis matches recurrence relation via master theorem.

[15] Include your raw and mean experimental outcomes, plot, and your discussion of the pattern in your plot:

	n = 10	n = 100	n = 1000	n = 10000	n = 100000	n = 500000	n = 1000000
Run #1	t = 0ms	t = 1ms	t = 9ms	t = 94ms	t = 956ms	t = 4724ms	t = 9804ms
Run #2	t = 0ms	t = 0ms	t = 10ms	t = 90ms	t = 960ms	t = 4703ms	t = 9737ms
Run #3	t = 0ms	t = 1ms	t = 9ms	t = 91ms	t = 945ms	t = 4663ms	t = 9651ms
Run #4	t = 0ms	t = 1ms	t = 9ms	t = 94ms	t = 948ms	t = 4628ms	t = 9812ms
Run #5	t = 0ms	t = 1ms	t = 10ms	t = 91ms	t = 944ms	t = 4656ms	t = 9676ms
Average	t = 0ms	t = 0.8ms	t = 9.4ms	t = 92ms	t = 951ms	t = 4675ms	t = 9736ms

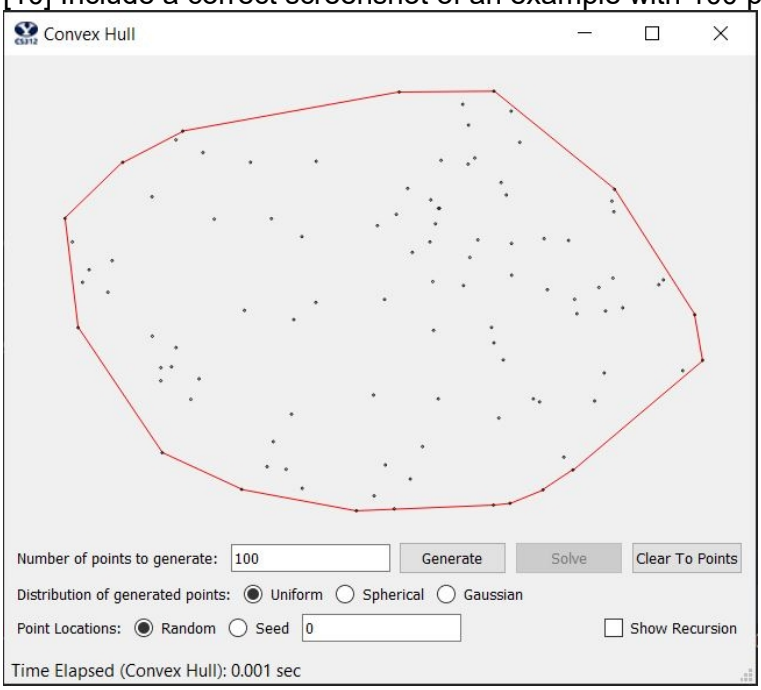
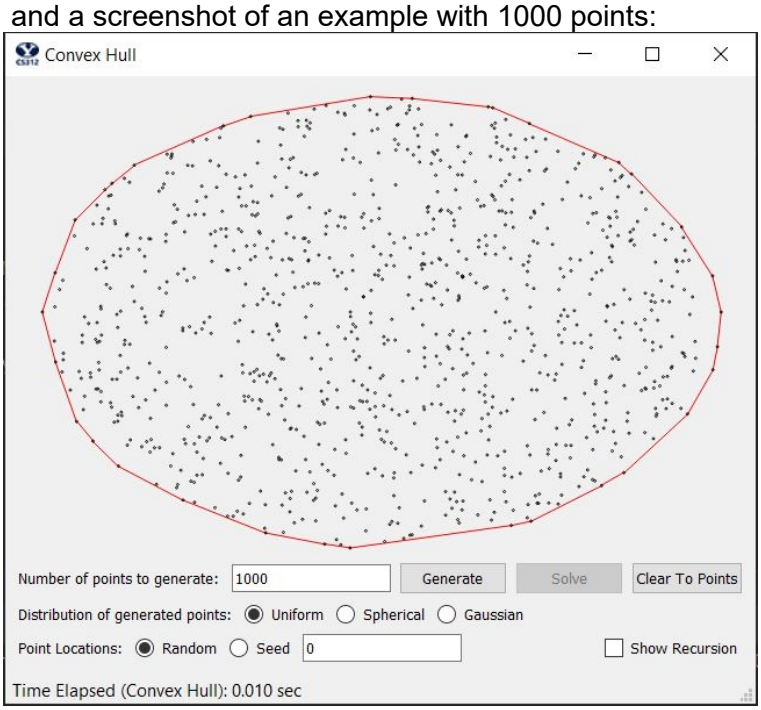
The plot of the line is just barely over linear growth, making it nlogn.
Plot equation is about $0.0005n\log_2 n$.

Which order of growth fits best?:
T(n) = O(nlogn).

Give an estimate of the constant of proportionality:
k = 0.0005.

[10] Discuss and explain your observations with your theoretical and empirical analyses, including any differences seen:
My theoretical analysis yielded O(nlogn), with the full equation being $O(4n\log_2 n)$. My empirical analysis yielded O(nlogn), with the full equation being around $O(0.0005n\log_2 n)$. Both analyses fall into the O(nlogn) order of growth. Both analyses share a full equation of the form $O(kn\log_2 n)$. The analyses differ on the k value. The difference in k value is likely due to the difference in units used for time.

[10] Include a correct screenshot of an example with 100 points:



and a screenshot of an example with 1000 points: